



University of Asia Pacific

Department of CSE

Assignment On

Comparison of Round Robin, SJF, and FCFS CPU Scheduling Algorithms

Course Title: Operating System Lab

Course Code: CSE 402

Submitted To

Atia Rahman Orthi

Lecturer

Department of Computer Science
and Engineering

University of Asia Pacific

Submitted By

Tashnimul Hasan Shifat

ID: 23101142

2022–2023 Batch

Department of CSE
University of Asia Pacific

Date of Submission: 17 August 2026

1. Problem:

Implement the following **CPU Scheduling Algorithms** for the given set of processes:

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	7
P2	1	4
P3	2	15
P4	3	11
P5	4	20
P6	4	9

Implement the following scheduling algorithms:

- **Round Robin (RR)** with **Time Quantum = 5**
- **Shortest Job First (SJF) – Non-Preemptive**
- **First Come First Serve (FCFS)**

For each scheduling algorithm, calculate:

- **Completion Time (CT)**
- **Turnaround Time (TAT)**
- **Waiting Time (WT)**
- **Average Turnaround Time (AVG TAT)**
- **Average Waiting Time (AVG WT)**

Also, represent the **process execution sequence/Gantt Chart** for each algorithm.

Finally, **compare the Average TAT and Average WT** of Round Robin, SJF, and FCFS and determine which scheduling algorithm performs better for the given set of processes. Explain the reason for the difference in performance.

2. Source Code:

```
43     is_done = [False] * n
44
45     while completed < n:
46         ready = [i for i in range(n) if procs[i]["at"] <= curr_time and not is_done[i]]
47
48         if not ready:
49             curr_time += 1
50             continue
51
52         idx = min(ready, key=lambda i: procs[i]["bt"])
53
54         curr_time += procs[idx]["bt"]
55         procs[idx]["ct"] = curr_time
56         procs[idx]["tat"] = procs[idx]["ct"] - procs[idx]["at"]
57         procs[idx]["wt"] = procs[idx]["tat"] - procs[idx]["bt"]
58
59         seq.append(procs[idx]["id"])
60         is_done[idx] = True
61         completed += 1
62
63     avg_tat = sum(p["tat"] for p in procs) / n
64     avg_wt = sum(p["wt"] for p in procs) / n
65     return seq, procs, avg_tat, avg_wt
66
67
68 def schedule_round_robin(proc_data, q=2):
69     procs = [p.copy() for p in sorted(proc_data, key=lambda x: x["at"])]
70     n = len(procs)
71     rem_bt = [p["bt"] for p in procs]
72
73     curr_time = 0
74     completed = 0
75     seq = []
76     ready_q = deque()
77     in_queue = [False] * n
78
79     while completed < n:
80         for i in range(n):
81             if procs[i]["at"] <= curr_time and not in_queue[i]:
82                 ready_q.append(i)
```

```

83         in_queue[i] = True
84
85     if not ready_q:
86         curr_time += 1
87         continue
88
89     idx = ready_q.popleft()
90     seq.append(procs[idx]["id"])
91     exec_time = min(rem_bt[idx], q)
92
93     rem_bt[idx] -= exec_time
94     curr_time += exec_time
95
96     for i in range(n):
97         if procs[i]["at"] <= curr_time and not in_queue[i]:
98             ready_q.append(i)
99             in_queue[i] = True
100
101     if rem_bt[idx] > 0:
102         ready_q.append(idx)
103     else:
104         procs[idx]["ct"] = curr_time
105         procs[idx]["tat"] = procs[idx]["ct"] - procs[idx]["at"]
106         procs[idx]["wt"] = procs[idx]["tat"] - procs[idx]["bt"]
107         completed += 1
108
109     avg_tat = sum(p["tat"] for p in procs) / n
110     avg_wt = sum(p["wt"] for p in procs) / n
111     return seq, procs, avg_tat, avg_wt
112
113
114 dataset = [
115     {"id": "p1", "at": 0, "bt": 7},
116     {"id": "p2", "at": 1, "bt": 4},
117     {"id": "p3", "at": 2, "bt": 15},
118     {"id": "p4", "at": 3, "bt": 11},
119     {"id": "p5", "at": 4, "bt": 20},
120     {"id": "p6", "at": 4, "bt": 9},
121 ]

```

```

122
123
124 fcfs_res = schedule_fcfs(dataset)
125 sjf_res = schedule_sjf(dataset)
126 rr_res = schedule_round_robin(dataset, q=2)
127
128 display_results("FCFS", *fcfs_res)
129 display_results("SJF", *sjf_res)
130 display_results("Round Robin", *rr_res, quantum=2)
131
132 print(f"\n{'='*10} COMPARISON SUMMARY {'='*10}")
133 print(f"{'Algorithm':<15}{'Avg TAT':<12}{'Avg WT':<12}")
134 print("-" * 39)
135 print(f"{'FCFS':<15}{fcfs_res[2]:<12.2f}{fcfs_res[3]:<12.2f}")
136 print(f"{'SJF':<15}{sjf_res[2]:<12.2f}{sjf_res[3]:<12.2f}")
137 print(f"{'Round Robin':<15}{rr_res[2]:<12.2f}{rr_res[3]:<12.2f}")

```

3. Output:

... ===== FCFS =====
Execution Sequence: p1 -> p2 -> p3 -> p4 -> p5 -> p6

PID	AT	BT	CT	TAT	WT

p1	0	7	7	7	0
p2	1	4	11	10	6
p3	2	15	26	24	9
p4	3	11	37	34	23
p5	4	20	57	53	33
p6	4	9	66	62	53

Average Turnaround Time = 31.67
Average Waiting Time = 20.67

===== SJF =====
Execution Sequence: p1 -> p2 -> p6 -> p4 -> p3 -> p5

PID	AT	BT	CT	TAT	WT

p1	0	7	7	7	0
p2	1	4	11	10	6
p3	2	15	46	44	29
p4	3	11	31	28	17
p5	4	20	66	62	42
p6	4	9	20	16	7

Average Turnaround Time = 27.83
Average Waiting Time = 16.83

===== Round Robin =====
Time Quantum: 2
Execution Sequence: p1 -> p2 -> p3 -> p1 -> p4 -> p5 -> p6 -> p2 -> p3 -> p1 -> p4 -> p5 -> p6 -> p3 -> p1 -> p4 -> p5 -> p6 -> p3 -> p4 -> p5 -> p6 -> p3

PID	AT	BT	CT	TAT	WT

p1	0	7	29	29	22
p2	1	4	16	15	11
p3	2	15	60	58	43
p4	3	11	53	50	39
p5	4	20	66	62	42
p6	4	9	50	46	37

Average Turnaround Time = 43.33
Average Waiting Time = 32.33

===== COMPARISON SUMMARY =====		
Algorithm	Avg TAT	Avg WT

FCFS	31.67	20.67
SJF	27.83	16.83
Round Robin	43.33	32.33

4. Discussion:

This experiment evaluated three CPU scheduling techniques: Round Robin (RR), Shortest Job First (SJF), and First Come First Serve (FCFS), using an identical set of six processes. Their performances were analyzed by comparing Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT).

Among the three methods, SJF produced the most efficient results for the given workload. It achieved an Average TAT of **27.83 units** and an Average WT of **16.83 units**, both of which were the lowest. FCFS showed comparatively moderate results, with an Average TAT of **31.67 units** and an Average WT of **20.67 units**. On the other hand, Round Robin, using a time quantum of 5 units, resulted in an Average TAT of **42.17 units** and an Average WT of **31.17 units**.

The lower waiting and turnaround times of SJF indicate that selecting shorter processes first can improve CPU scheduling efficiency for this particular process set. FCFS is simpler because processes are handled according to their arrival order, but longer processes can cause other processes to wait. RR ensures that each process receives regular CPU time, making it suitable for interactive and time-sharing environments; however, the repeated context switching and time-slice allocation led to higher waiting and turnaround times in this experiment.

5. Conclusion:

The experiment demonstrated the performance differences between RR, SJF, and FCFS scheduling algorithms. Based on the calculated CT, TAT, and WT values, **SJF provided the best performance for the selected processes**, achieving the lowest Average TAT (**27.83**) and Average WT (**16.83**).

FCFS ranked second, with an Average TAT of **31.67** and an Average WT of **20.67**, while RR produced the highest values, with an Average TAT of **42.17** and an Average WT of **31.17**. Although RR has the advantage of providing fair CPU time to every process, it was not the most efficient method for this particular workload. Therefore, when the primary objective is to reduce waiting and turnaround time for these processes, **SJF is the most suitable scheduling algorithm among the three tested methods**.

6. GitHub: